

Факултет техничких наука • Нови Сад

# Паралелно генерисање фракталних стабала: поређење Python и Rust имплементација

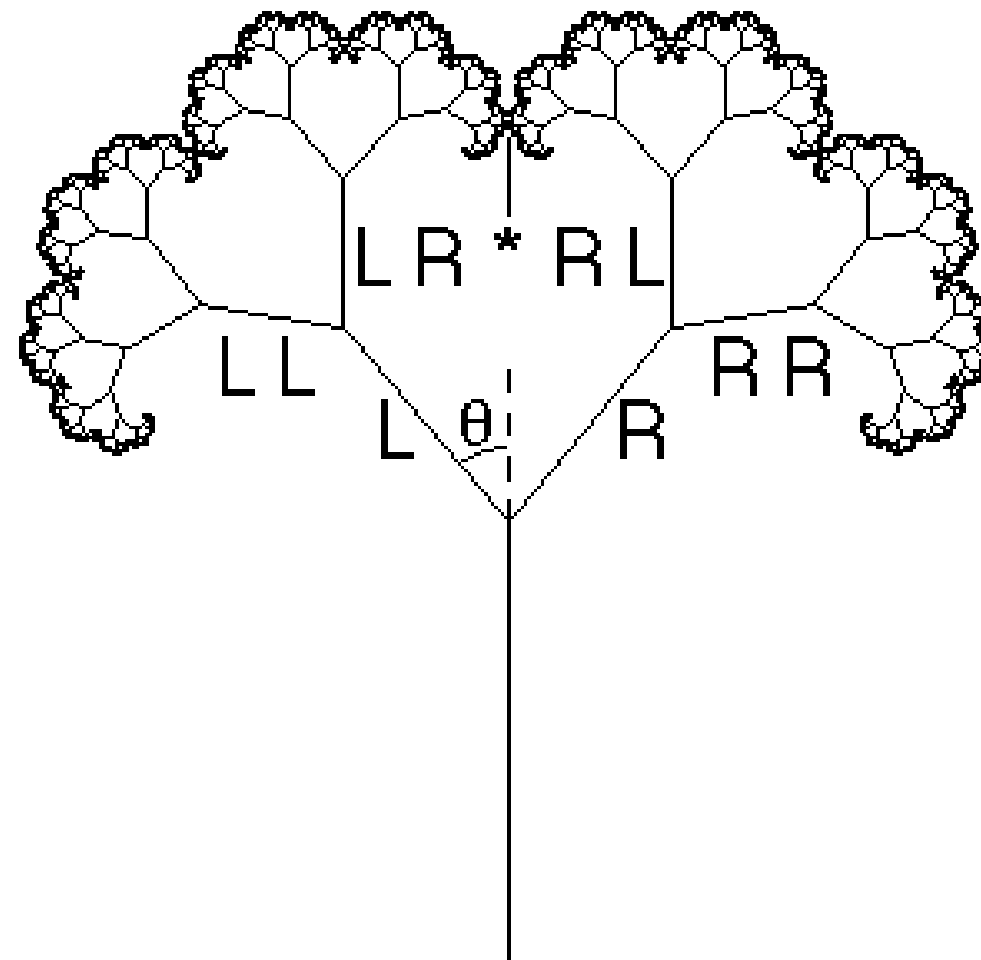
---

Ана Попарић • SV 74/2021

Ментор: проф. Игор Дејановић

Софтверско инжењерство и информационе технологије

2026



# Фрактално стабло — проблем и мотивација



- Самосличност — свако подстабло је умањена копија целог стабла
- Рачунски интензиван (*CPU-bound*) задатак
- Python — једноставност
- Rust — перформансе

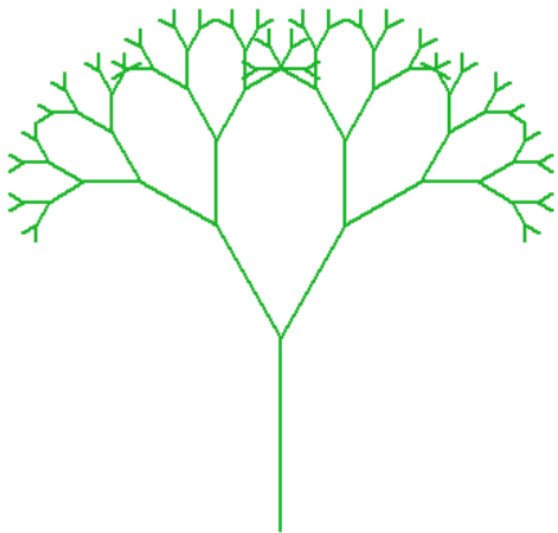
## Симетрично стабло

|              |               |
|--------------|---------------|
| $l_{\min}$   | 0.01          |
| Број грана   | 8 388 607     |
| Python (N=1) | <b>8,0 s</b>  |
| Rust (N=1)   | <b>0,20 s</b> |

## Асиметрично стабло

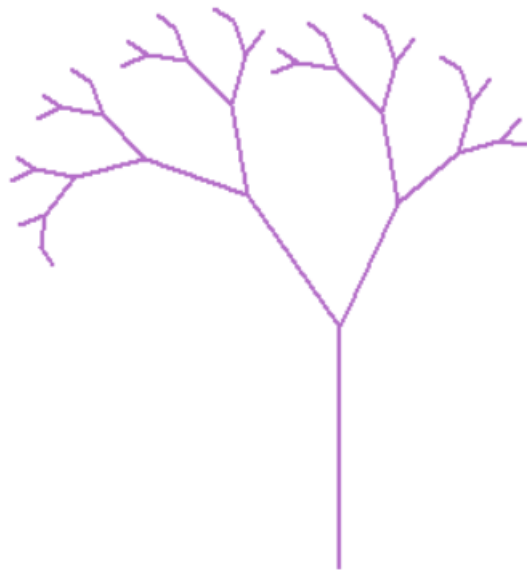
|              |               |
|--------------|---------------|
| $l_{\min}$   | 0.0023        |
| Број грана   | 8 464 173     |
| Python (N=1) | <b>8,2 s</b>  |
| Rust (N=1)   | <b>0,25 s</b> |

## Типови стабла и стратегија паралелизације



Симетрично

$$r_l = r_d = 0.67$$



Асиметрично

$$r_l = 0.67,$$
$$r_d = 0.57$$

- $\text{split\_depth} = \lceil \log_2(N \times 4) \rceil$
- Фактор 4  $\rightarrow$  *oversubscription*

## Python и Rust — различити механизми паралелизације

### Python

GIL → паралелизација **процесима**

`Pool(processes=N) + pool.map`

`spawn`: нов интерпретер по процесу

`pickle`: серијализација

Статичка расподела

### Rust

Ownership → паралелизација **нитима**

`rayon::ThreadPool + par_iter()`

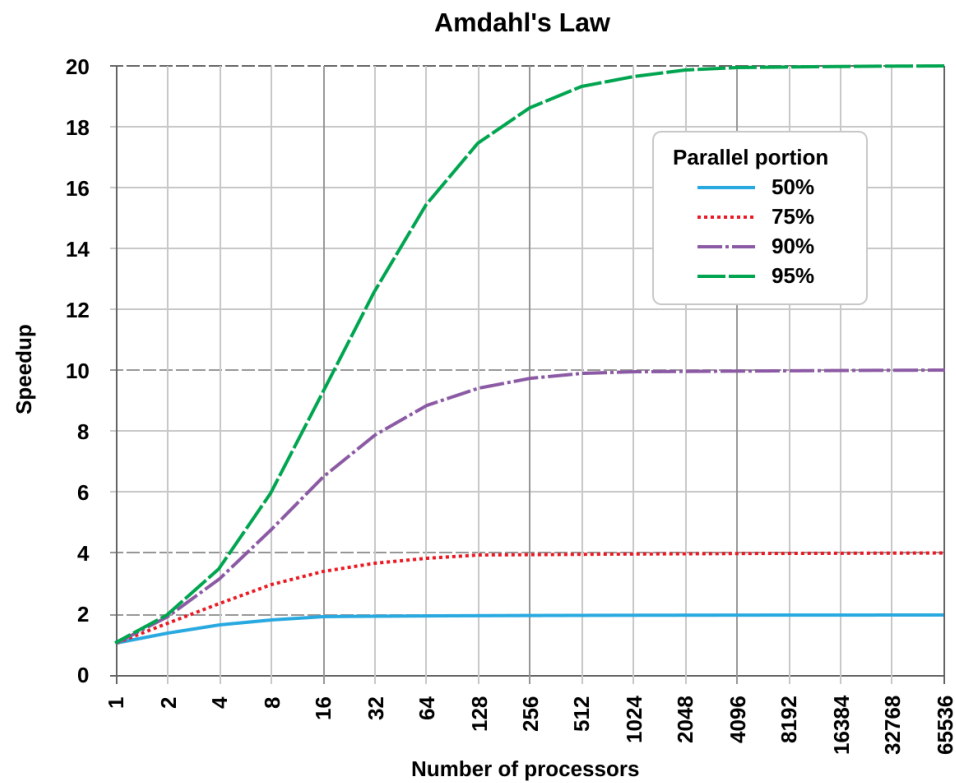
Нити деле меморијски простор

Без серијализације

**Крађа посла** (work-stealing)

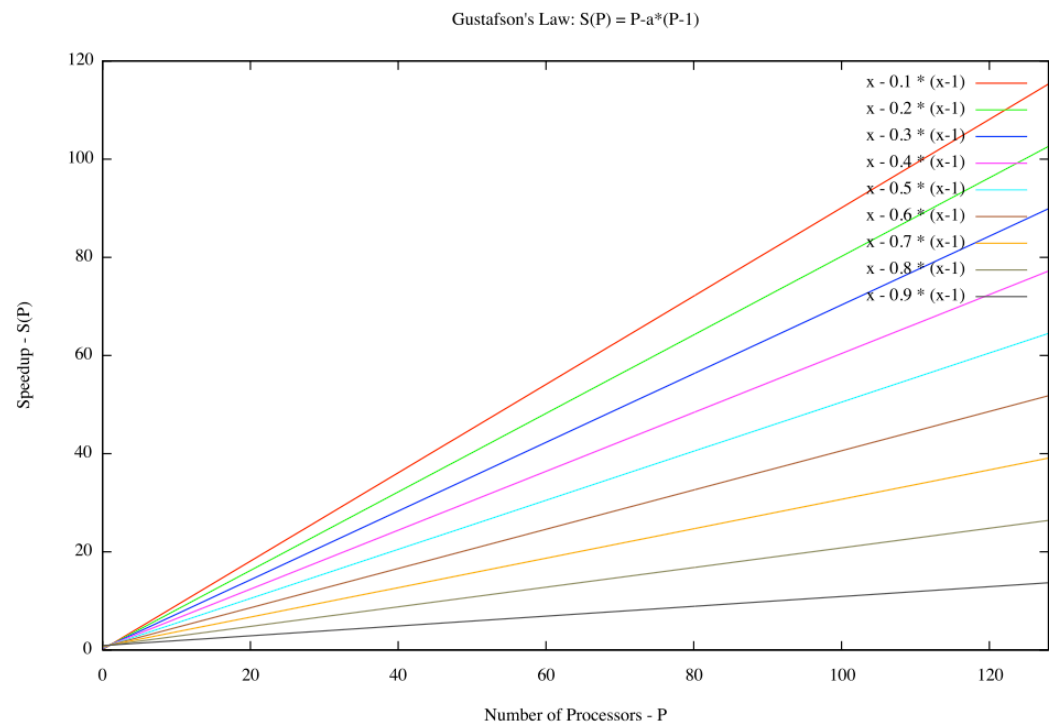
| Процесор             | Језгра (физ./лог.)      | Меморија  | ОС             |
|----------------------|-------------------------|-----------|----------------|
| Intel Core i5-1035G1 | 4 / 8 (Hyper-Threading) | 8 GB DDR4 | Windows 11 Pro |

# Теоријски оквир: Амдалов и Густафсонов закон



Јако скалирање — фиксан проблем

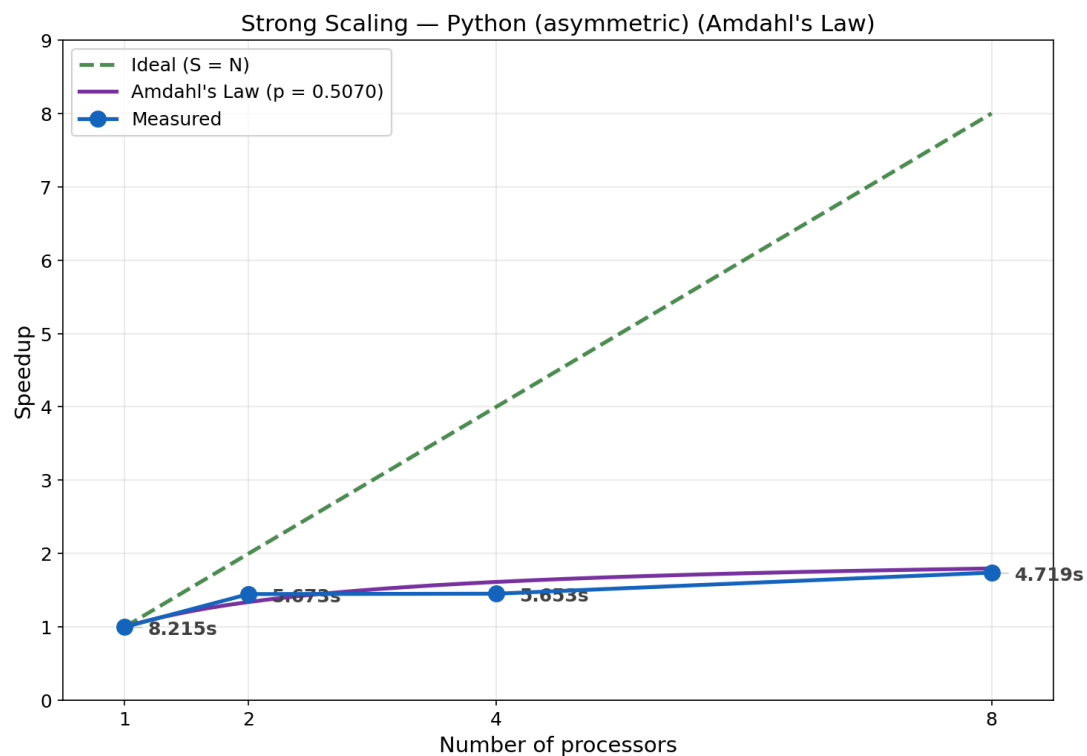
$$S_{\max} = \frac{1}{f + \frac{1-f}{N}}$$



Слабо скалирање — растући проблем

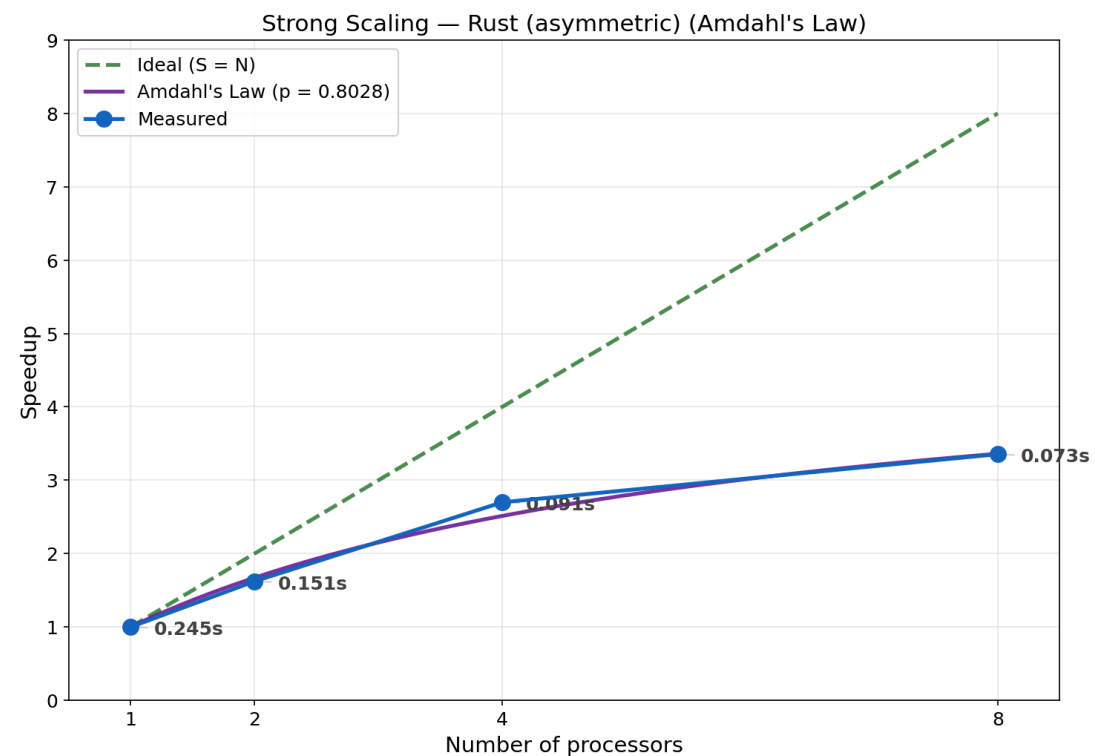
$$S_{\text{scaled}} = N + (1 - N) \times f$$

# Јако скалирање — асиметрично стабло



Python

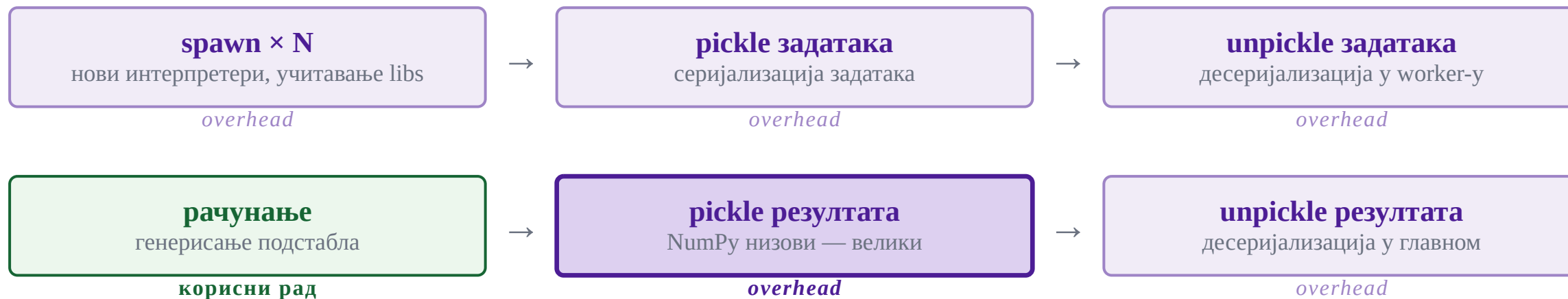
Стагнација при  $N = 4$  — статичка расподела, неједнака подстабла



Rust

$S = 3.355 \approx$  Амдал 3.361 — крађа посла надокнађује асиметрију

## Зашто Python не скалира? — spawn + pickle overhead



| $N$ | $T(N)$ (s)   |
|-----|--------------|
| 4   | 3,872        |
| 8   | <b>4,294</b> |

- $N = 8$  спорији од  $N = 4$  — *overhead* надмашује корист
- $N = 8 \rightarrow 8$  нових интерпретера + 8 десеријализација
- Rust: дељена меморија  $\rightarrow$  *overhead* занемарљив

# Закључак

## Главни налази

1. Механизам паралелизације је одлучујући
2. `split_depth` стратегија
3. *Hyper-Threading*

## Даљи рад:

- `shared_memory` уместо `pickle`
- `fork` метода (Linux)
- Платформа без *Hyper-Threading*

## Убрзање при $N = 8$

| $N = 8$                       | Python | Rust         |
|-------------------------------|--------|--------------|
| Симетрично ( $S$ )            | 1.853  | <b>3.244</b> |
| Асиметрично ( $S$ )           | 1.741  | <b>3.355</b> |
| Сим. ( $S_{\text{scaled}}$ )  | 1.045  | <b>3.168</b> |
| Асим. ( $S_{\text{scaled}}$ ) | 1.154  | <b>3.878</b> |



**Хвала на пажњи**

**Питања?**